

第34讲：Open-source / Open-weight LLM

模型权重、本地部署和 API 到底有什么区别？——Checkpoint、Hugging Face、Safetensors、GGUF、Transformers、vLLM 与 llama.cpp

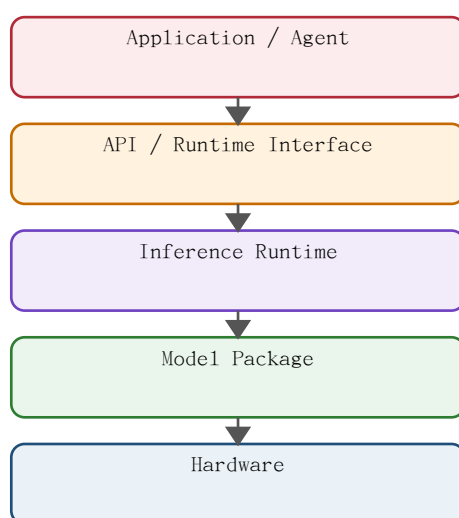
本讲目标

学完后你应该能解释：Architecture、Checkpoint、Weights、Tokenizer、Model Card

分别是什么；Hugging Face 模型仓库里为什么有很多文件；Safetensors 和 GGUF

分别解决什么问题；Transformers、vLLM、llama.cpp 是模型还是运行时；以及本地部署与商业 API 的系统边界到底在哪里。

本地部署 = 你自己负责整条栈；API = 供应商替你负责下面大部分层。



先记住一句：“模型”不是一个文件；真正能跑起来，需要 结构 + 权重 + tokenizer + 配置 + runtime + hardware。

1. 先澄清：Open-source 和 Open-weight 不完全一样

日常交流里大家常把“可以下载权重的模型”都叫开源模型，但严格来说，开放到什么程度取决于许可证、代码、训练数据和权重等多个层次。

概念	通俗理解
Open-weight	可以下载模型权重，但许可证和训练细节可能有限制
Open-source software	运行/训练代码以开源许可证发布
Open data	训练或评测数据公开
Fully reproducible	从数据、训练代码到权重都足以重建

科研提醒

不要只写“我们使用开源模型”。论文里最好写清具体 checkpoint、license/repo、revision 和运行方式。

2. Architecture vs Checkpoint：骨架和训练后的状态

Architecture 是模型的“骨架”：层数、hidden size、attention heads、激活函数等。Checkpoint 是某个架构在训练到某个状态后保存下来的参数。

Architecture	Checkpoint
定义计算图和结构	包含训练后的具体权重
同一架构可有多个 checkpoint	每个 checkpoint 行为可能不同
随机初始化时还不会任务	加载 checkpoint 后才有学到的能力

例子

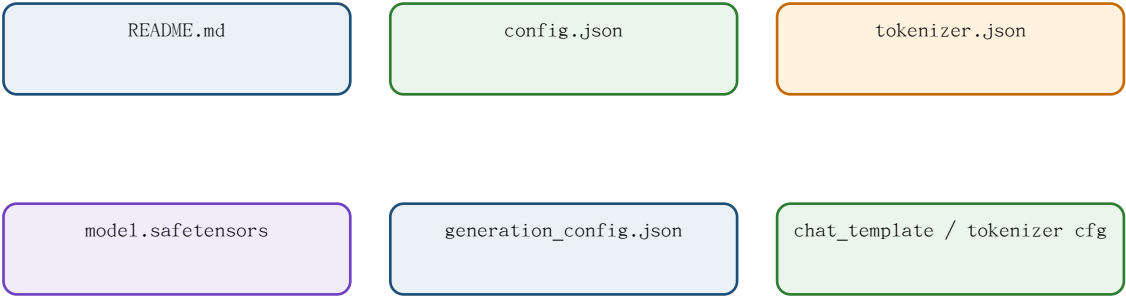
“Llama-style decoder architecture”像汽车底盘设计；某个训练完成的 7B checkpoint 像一辆真正调校过、装好发动机参数的车。

3. Weights 到底是什么？

Weights 就是训练后得到的大量张量：Embedding、Attention 投影、FFN 权重、Norm 参数等。这些张量决定了模型在给定输入时怎样做数值变换。因此下载模型权重，本质上是拿到模型“长期学到的参数状态”，而不是拿到训练数据本身。

4. 一个 Hugging Face Model Repo 里通常有什么？

一个 Model Repo 往往不只是一份“权重文件”



Weights + Architecture Config + Tokenizer + Generation/Chat Metadata + Model Card

缺任何一层，都可能导致“同一个模型”跑法不同。

常见文件包括:

文件	作用
model.safetensors / shards	模型权重张量
config.json	层数、hidden size、架构类型等结构参数
tokenizer.json / vocab / merges	文本怎样切成 token
tokenizer_config.json	tokenizer 行为和特殊 token
generation_config.json	默认生成参数
README.md / Model Card	模型用途、限制、训练/评测信息

5. 为什么 Tokenizer 也是模型的一部分？

同样一句文字，用不同 tokenizer 会得到不同 token IDs；而模型的 embedding table 是按训练时词表学出来的。

```
wrong tokenizer -> wrong token IDs -> wrong embeddings -> broken behavior
```

所以“我有权重”并不代表随便配一个 tokenizer 就能跑。

6. config.json 为什么重要？

Runtime 必须知道模型该搭成什么结构。config.json 往往告诉加载器：模型类型、层数、hidden dimension、head 数、vocab size 等。

加载流程可以抽象为：

```
read config -> build architecture -> load checkpoint tensors -> attach tokenizer -> generate
```

7. Safetensors 是什么？

Safetensors 是一种面向张量的序列化格式。它的目标之一是避免 pickle 类格式可能执行任意代码的风险，同时支持快速、按需读取张量。

特点	意义
只存张量 + metadata	更适合模型权重交换
不依赖 Python pickle 执行对象	安全边界更清晰
支持切片/部分加载	有利于大模型加载/分片
常见于 Hugging Face 模型仓库	研究工作流中非常常见

别误解

Safetensors 是“权重文件格式”，不是模型架构，也不是推理引擎。

8. `.bin` / pickle 为什么需要谨慎？

历史上很多 PyTorch 权重使用 pickle 相关格式。pickle 可以反序列化 Python 对象，也意味着不可信文件可能包含安全风险。

所以下载第三方模型文件时，除了模型效果，也要考虑来源和文件格式。

9. GGUF 是什么？

GGUF 是 llama.cpp / ggml 生态常用的模型文件格式。它不仅保存张量，还携带模型元数据，特别适合 llama.cpp 的本地推理、量化模型和跨硬件部署。

常见流程：

HF checkpoint -> convert to GGUF -> quantize (optional) -> llama.cpp runtime
通俗理解 Safetensors 更像通用深度学习权重包装；GGUF 更像为 ggml/llama.cpp 推理生态设计的“可直接上车”的模型包。

10. Quantized GGUF 文件名怎么看？

你会经常看到 Q4、Q5、Q8 等量化标签。它们大致表示权重使用更低比特表示，以降低内存与带宽需求。但具体命名方案和量化方法会随工具链演进，不能只看“4 bit”三个字就认为所有 Q4 完全一样。

科研复现 论文若使用 GGUF 量化模型，应写清具体文件名/quant scheme，而不是只写“4-bit model”。
--

11. Hugging Face Hub 到底是什么？

Hugging Face Hub 可以理解成一个版本化模型/数据/应用仓库平台。模型仓库可以包含 checkpoint、config、tokenizer、model card 等文件，并可通过 revision / tag / commit 固定版本。对科研最重要的是：你可以把模型版本固定到具体 revision，而不是永远使用“main 最新版”。

12. `from\_pretrained()` 到底在做什么？

在 Transformers 里，常见代码是：

```
tokenizer = AutoTokenizer.from_pretrained(repo_id) model =  
AutoModelForCausalLM.from_pretrained(repo_id)
```

背后会完成：下载/读取配置、构造模型结构、加载 checkpoint、加载 tokenizer 和相关配置。

注意

`from\_pretrained()` 不是“在线调用 API”；它通常是在本地 Python 进程里把模型真正加载到 CPU/GPU 内存。

13. Transformers 是什么？

Transformers 是 Hugging Face 的通用模型库，适合研究、训练、微调和本地推理。它提供统一的 `AutoModel` / `AutoTokenizer` / `pipeline` 等接口。

它的重点是“易用和广泛模型支持”，而不是专门追求大规模在线 `serving` 的最高吞吐。

14. vLLM 是什么？

vLLM 更偏向高吞吐 LLM `serving`。你可以把模型加载进 vLLM，然后启动 HTTP server，让上层应用像调用 API 一样请求本地/自托管模型。

当前官方文档提供 OpenAI-compatible server，可暴露 Chat、Completions、Responses、Embeddings 等接口。

底层联系
第22讲讲过的 KV Cache、batching、serving scheduler，正是这类推理引擎重点优化的对象。

15. llama.cpp 是什么？

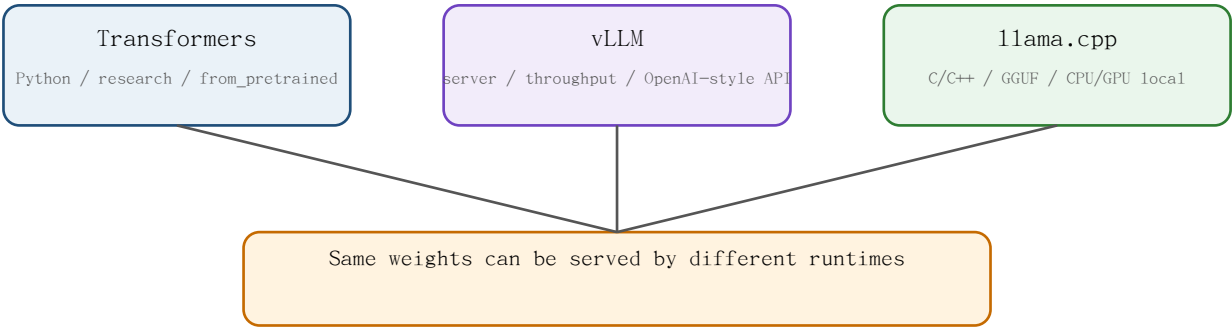
llama.cpp 是 C/C++ 推理项目，目标是让 LLM 在广泛硬件上以较低依赖运行，包括 CPU、Apple Silicon、NVIDIA GPU 等。

它常使用 GGUF 文件，并支持多种低比特量化与 CPU+GPU 混合推理。

因此它尤其适合：本地桌面、笔记本、CPU-only、小显存和便携部署。

16. Transformers、vLLM、llama.cpp 不是同一层东西

三者不是“模型”，而是不同推理/加载工具链



Runtime 会影响吞吐、延迟、支持格式、量化方式与复现性。

工具	更像什么	典型使用
Transformers	通用 Python 模型库	研究、训练、微调、本地推理
vLLM	高吞吐 serving engine	GPU 服务器、自托管 API
llama.cpp	轻量跨硬件 inference runtime	桌面/CPU/GPU 本地运行、GGUF

17. 同一权重，用不同 Runtime 会完全一样吗？

不一定。即使 checkpoint 相同，以下因素也可能让结果不同：

变量	影响
Tokenizer / chat template	输入 token 序列不同
Quantization	数值近似不同
Kernel / precision	数值误差可能不同
Sampling implementation	随机路径可能不同
Default generation config	temperature/top-p/max tokens 不同
Runtime version	实现与 bugfix 会变化

18. 本地部署到底是什么意思？

本地部署意味着：模型权重、推理 runtime、KV Cache、GPU/CPU 内存、进程生命周期和数据都由你自己的机器/服务器管理。

```
Local request -> your runtime -> your weights -> your hardware -> result
```

因此你能控制模型文件、版本、量化、系统 prompt、日志和数据路径，但也要自己负责安装、显存、性能和安全。

19. 调用 API 到底意味着什么？

API 模式下，你发送 token/文本请求给模型提供方，对方负责：

供应商负责	你负责
模型权重和部署	Prompt / request
GPU 集群	采样参数
KV Cache / batching	应用逻辑
服务扩缩容	评测与记录
模型升级和运行时	遵守 API 契约

你通常看不到真实 checkpoint 文件，也无法控制底层 runtime。

20. API vs Local: 最实际的比较

维度	商业/API	本地/自托管
上手	快	要部署
模型能力	可直接用最强闭源模型	取决于可下载权重和硬件
成本	按 token / request	硬件 + 电费 + 运维
隐私	数据离开本地边界（取决于服务政策）	可完全本地
可控性	底层有限	权重/runtime/量化可控
可复现性	供应商版本可能变化	可 pin checkpoint/runtime
大规模实验	方便但 token 成本累积	硬件固定后边际成本低

对科研者
API 很适合快速实验和使用强模型；本地权重更适合需要严格控制版本、量化、内部日志或大量重复运行的实验。两者不是互斥关系。

21. “下载一个模型” 到底下载多少东西？

一个完整模型仓库可能包含多个权重 shard、tokenizer、config 和文档。比如权重太大时，会拆成多个 ``.safetensors`` 文件，再用 `index` 文件记录每个 tensor 在哪个 shard。

因此下载大小不等于单个文件大小。

22. Model Revision: 为什么科研实验要固定 commit?

模型仓库的 ``main`` 可能更新。若半年后仓库更新了 tokenizer、chat template 或权重，你再运行 ``from_pretrained(repo_id)``，得到的环境可能已不同。

所以科研应记录：

`repo_id + revision/commit hash + runtime version + tokenizer + chat template`

复现性
“模型名字一样”并不保证“实验对象一样”。Checkpoint version 必须进入实验记录。

23. Model Card: 为什么不是可有可无？

Model Card 通常描述模型用途、限制、训练信息、评测结果、许可证和已知风险。

下载权重前，先读 Model Card，尤其要确认：模型类型、chat template、许可证、量化说明、上下文长度、适用/不适用场景。

24. Chat Template: 同一模型为什么 Prompt 格式很重要？

聊天模型通常不是直接把“用户文字”裸送进去，而是转成训练时熟悉的特殊角色格式。

`system/user/assistant -> chat template -> token sequence`

模板错误会让同一 checkpoint 的表现明显下降，尤其是 Tool Use / Agent 实验。

25. 本地 Serving: 为什么要启动一个 API Server?

如果只有一个 Python 脚本, 你可以直接在进程里调用模型; 但多个应用/Agent 需要共享一个模型时, 更常见做法是把模型运行时包装成 HTTP server。

```
Agent processes -> local OpenAI-style API -> vLLM/llama.cpp server -> model
```

这样上层代码不需要关心底层权重加载。

26. 研究中最容易忽略的变量：Runtime

在 Agent Systems 论文里，大家常写“使用 Model X”，但模型实际行为还受 runtime 影响。

必须记录	例子
Runtime	Transformers / vLLM / llama.cpp
Version	具体 package / commit
Precision	BF16 / FP16 / INT8 / GGUF Q4
Device	GPU/CPU 型号和数量
Context length	最大上下文设置
Generation	temperature/top-p/max tokens/seed
Chat template	具体模板

27. 本地模型与 Agent 科研：为什么特别有价值？

优势	研究意义
固定 checkpoint	减少模型升级带来的漂移
可重复大量调用	适合 repeated trials
可做量化/LoRA	能操纵机制变量
可控制 runtime	便于系统实验
可记录内部 serving 指标	更容易测 latency/KV/cost

同时别神化
本地模型不是天然更科学。如果模型太弱、环境配置不稳定、量化严重失真，也会让结论失去外部有效性。

28. API 模型与科研：为什么也完全可以用？

大量 Agent 研究会直接使用商业 API，因为它们模型能力强、部署成本低、上手快。

但要记录：

provider model id + date/version if available + sampling params + repeated trials + total tokens/cost

还要承认一个限制：供应商内部模型版本和 serving 细节可能无法完全固定。

29. 一个实用选择框架

目标	更合适的起点
先验证研究现象	API
控制模型版本/量化	本地权重
需要最强能力	强商业 API

目标	更合适的起点
大量重复实验	本地或低价 API
研究 serving / KV / batching	本地自托管
研究 LoRA / 量化异质性	本地权重
需要私有数据不出机器	本地

30. 小实验：本地部署内存估算器

材料包中的 Python 实验输入模型参数数量和数值格式，估算“纯权重”内存，并对比 1B / 7B / 14B / 32B 在 BF16、INT8、理想 INT4 下的数量级。

同时实验会生成一张“部署栈检查表”，提醒你：

weights -> tokenizer/config -> runtime -> device -> context/KV -> app/API

为什么只是估算
真实运行还要加 KV Cache、workspace、量化 metadata、runtime buffer 等；不能把权重大小直接等同于总显存需求。

31. 科研复现清单

类别	记录内容
Model	repo_id / checkpoint / revision
Files	safetensors / GGUF / quantization
Tokenizer	版本 / special tokens
Chat	chat template / system prompt
Runtime	Transformers/vLLM/llama.cpp + version
Hardware	GPU/CPU/VRAM/RAM
Inference	context, batch, temperature, top-p, seed
Serving	concurrency / batching / API server config

以后写论文
只写 “we use Qwen/Llama/DeepSeek” 远远不够。模型本身、文件、runtime、模板和推理参数共同定义了你的实验对象。

32. 常见误区汇总

误区	更准确的理解
模型就是一个 .safetensors 文件	还需要 config、tokenizer、runtime 等
Checkpoint = Architecture	checkpoint 是特定权重状态
Safetensors = 推理引擎	只是张量文件格式

误区	更准确的理解
GGUF = 模型架构	是 ggml/llama.cpp 生态模型格式
Transformers/vLLM/llama.cpp 是模型	它们是库/推理 runtime
API = 模型在我电脑上运行	实际计算发生在提供方服务器
同模型名一定完全复现	revision、runtime、quantization、template 都会影响

33. 当前软件生态备注（截至 2026-09）

为避免软件快速变化带来的混淆，本讲只保留较稳定的层次关系。以下是当前官方文档所确认的几个事实：

组件	当前官方文档确认的定位
Hugging Face Hub	版本化模型仓库，可存 checkpoint、config 等文件
Safetensors	安全、快速的张量存储格式
Transformers	可通过 AutoTokenizer / AutoModelForCausalLM 加载模型
vLLM	提供面向 serving 的 OpenAI-compatible HTTP server
llama.cpp	跨硬件 C/C++ LLM inference，常使用 GGUF
GGUF	ggml/llama.cpp 生态模型格式，可携带张量和 metadata

软件具体命令、支持模型和接口会持续更新，真实实验前应检查对应版本的官方文档。

34. 练习题

- 1) Architecture、Checkpoint、Weights 三者有什么区别？
- 2) 为什么 Tokenizer 必须和模型匹配？
- 3) Safetensors 与 GGUF 分别是什么层次的东西？
- 4) Transformers、vLLM、llama.cpp 的定位有什么不同？
- 5) 为什么 `from_pretrained()` 不等于调用商业 API？
- 6) 本地部署与 API 的系统边界有什么不同？
- 7) 为什么模型 revision / commit 对科研复现很重要？
- 8) 为什么同一 checkpoint 在不同 runtime 下可能略有不同？
- 9) 做 Agent Systems 论文时，本地模型实验至少应记录哪些部署变量？
- 10) 什么情况下你会优先用 API，什么情况下优先用本地模型？

答案见材料包《03_第34讲_练习题与答案.pdf》。

35. 本讲总结

一句话总纲
一个可运行的本地 LLM 不是“一个文件”，而是一条完整栈：Architecture/Config + Checkpoint/Weights + Tokenizer/Chat Template + Runtime + Hardware + Inference Settings。API 只是把这条栈的大部分托管给供应商。

对象	最短记法
Architecture	模型骨架
Checkpoint	训练后的参数状态
Safetensors	通用张量权重格式
GGUF	llama.cpp/ggml 生态模型格式
Transformers	通用 Python 模型库
vLLM	高吞吐 serving runtime
llama.cpp	轻量跨硬件本地 inference
API	远程托管模型服务

下一讲：第35讲 《LLM 实验可复现性：为什么同一个 Prompt 重跑结果会变？》。